


[_ \(http://hl7.org/fhir/R4/searchform.html\)](http://hl7.org/fhir/R4/searchform.html)

[Exchange \(exchange-module.html\)](#) > **RESTful API**

This page is part of the FHIR Specification (v4.0.1: R4 - Mixed [Normative](https://confluence.hl7.org/display/HL7/HL7+Balloting) (<https://confluence.hl7.org/display/HL7/HL7+Balloting>) and [STU](https://confluence.hl7.org/display/HL7/HL7+Balloting) (<https://confluence.hl7.org/display/HL7/HL7+Balloting>)) in it's permanent home (it will always be available at this URL). The current version which supercedes this version is [5.0.0](http://hl7.org/fhir/index.html) (<http://hl7.org/fhir/index.html>). For a full list of available versions, see the [Directory of published versions](http://hl7.org/fhir/directory.html) (<http://hl7.org/fhir/directory.html>). Page versions: [R5](http://hl7.org/fhir/R5/http.html) (<http://hl7.org/fhir/R5/http.html>), [R4B](http://hl7.org/fhir/R4B/http.html) (<http://hl7.org/fhir/R4B/http.html>), [R4 R3](http://hl7.org/fhir/STU3/http.html) (<http://hl7.org/fhir/STU3/http.html>), [R2](http://hl7.org/fhir/DSTU2/http.html) (<http://hl7.org/fhir/DSTU2/http.html>).

3.1.0 RESTful API

[FHIR Infrastructure](http://www.hl7.org/Special/committees/fiwig/index.cfm)
[_ \(http://www.hl7.org/Special/committees/fiwig/index.cfm\)](http://www.hl7.org/Special/committees/fiwig/index.cfm) Work Group

[Maturity Level](#)
[_ \(versions.html#maturity\):](#) Normative

[Standards Status \(versions.html#std-process\):](#)
[Normative \(versions.html#std-process\)](#)


This page has been approved as part of an [ANSI](https://www.ansi.org/) (<https://www.ansi.org/>) standard. See the [Infrastructure \(ansi-infrastructure.html\)](#) Package for further details.

FHIR is described as a 'RESTful' specification based on common industry level use of the term REST. In practice, FHIR only supports Level 2 of the [REST Maturity model](https://martinfowler.com/articles/richardsonMaturityModel.html) (<https://martinfowler.com/articles/richardsonMaturityModel.html>) as part of the core specification, though full Level 3 conformance is possible through the use of [extensions](#) ([extensibility.html](#)). Because FHIR is a standard, it relies on the standardization of resource structures and interfaces. This may be considered a violation of REST principles but is key to ensuring consistent interoperability across diverse systems.

Each "resource type" has the same set of interactions defined that can be used to manage the resources in a highly granular fashion. Applications claiming conformance to this framework claim to be conformant to "RESTful FHIR" (see [Conformance \(conformance-rules.html\)](#)).

Note that in this RESTful framework, transactions are performed directly on the server resource using an HTTP request/response. The API does not directly address authentication, authorization, and audit collection - for further information, see the [Security Page \(security.html\)](#). All the interactions are all described for synchronous use, and an [Asynchronous use pattern \(async.html\)](#) is also defined.

The API describes the FHIR resources as a set of operations (known as "interactions") on resources where individual resource instances are managed in collections by their type. Servers can choose which of these interactions are made available and which resource types they support. Servers SHALL provide a [Capability Statement \(capabilitystatement.html\)](#) that specifies which interactions and resources are supported.

In addition to a number of [General Considerations](#) this page defines the following interactions:

Instance Level Interactions

read	Read the current state of the resource
vread	Read the state of a specific version of the resource
update	Update an existing resource by its id (or create it if it is new)
patch	Update an existing resource by posting a set of changes to it
delete	Delete a resource
history	Retrieve the change history for a particular resource

Type Level Interactions

create	Create a new resource with a server assigned id
search	Search the resource type based on some filter criteria
history	Retrieve the change history for a particular resource type

Whole System Interactions

<u>capabilities</u>	Get a capability statement for the system
<u>batch/transaction</u>	Update, create or delete a set of resources in a single interaction
<u>history</u>	Retrieve the change history for all resources
<u>search</u>	Search across all resource types based on some filter criteria

In addition to these interactions, there is an [operations framework \(operations.html\)](#), which includes endpoints for [validation \(operation-resource-validate.html\)](#), [messaging \(messaging.html#mailbox\)](#) and [Documents \(documents.html#bundle\)](#). Also, implementers can [use GraphQL \(graphql.html\)](#).

3.1.0.1 General Considerations

Note: Where the FHIR specification does not specify behavior with regards to HTTP capabilities (such as OPTIONS), implementers cannot expect greater consistency than is mandated in the underlying HTTP protocol.

3.1.0.1.1 Style Guide

The interactions on this page are defined like this:

```
VERB [base]/[type]/[id] {?_format=[mime-type]}
```

- The first word is the HTTP verb used for the interaction
- Content surrounded by [] is mandatory, and will be replaced by the string literal identified. Possible insertion values:
 - **base** : The [Service Base URL](#)
 - **mime-type** : The [Mime Type](#)
 - **type** : The name of a resource type (e.g. "Patient")
 - **id** : The [Logical Id \(resource.html#id\)](#) of a resource
 - **vid** : The [Version Id \(resource.html#metadata\)](#) of a resource
 - **compartment** : The name of a [compartment \(compartmentdefinition.html\)](#)
 - **parameters** : URL parameters as defined for the particular interaction
- Content surrounded by {} is optional

Implementations constructing URLs using these patterns SHOULD conform to [RFC 3986 Section 6 Appendix A](https://tools.ietf.org/html/rfc3986#appendix-A) <https://tools.ietf.org/html/rfc3986#appendix-A> which requires percent-encoding for a number of characters that occasionally appear in the URLs (mainly in search parameters).

This specification uses the underscore as a prefix to disambiguate reserved names from other names in 3 cases:

- To differentiate system wide history and search interactions from interactions on Resource Types
- To differentiate search, history and similar interactions from instances of a resource type
- To differentiate search parameters defined for all resources from those defined for specific resource types

In addition, the character \$ is used as a prefix to operation names that are RPC-like additions to the base API defined either by this specification or by implementers.

3.1.0.1.2 Service Base URL

The Service Base URL is the address where all of the resources defined by this interface are found. The Service Base URL takes the form of

```
http{s}://server{/path}
```

The path portion is optional, and does not include a trailing slash. Each resource type defined in this specification has a manager (or "entity set") that lives at the address `/[type]` where the `[type]` is the name of the resource type. For instance, the resource manager for the type **Patient** will live at:

```
https://server/path/Patient
```

All the logical interactions are defined relative to the service root URL. This means that if the address of any one FHIR resource on a system is known, the address of other resources may be determined.

Note: All URLs (and ids that form part of the URL) defined by this specification are case sensitive. Clients SHOULD encode URLs using UTF-8, and servers SHOULD decode them assuming they are UTF-8 (for background, [see here](http://stackoverflow.com/questions/912811/what-is-the-proper-way-to-url-encode-unicode-characters)  (<http://stackoverflow.com/questions/912811/what-is-the-proper-way-to-url-encode-unicode-characters>)).

Note that a server may use a path of the form `http://server/...[xx]...` where the `[xx]` is some variable portion that identifies a particular instantiation of the FHIR API. Typically, the variable id identifies a patient or a user, and the underlying information is completely compartmented by the logical identity associated with `[xx]`. In this case, the FHIR API presents a patient or user centric view of a record, where authentication/authorization is explicitly granted to the URL, on the grounds that some identifiable user is associated with the logical identity. It is not necessary to explicitly embed the patient id in the URL - implementations can associate a FHIR end-point with a particular patient or provider by using an OAuth login. See [Compartment \(compartmentdefinition.html\)](#) for the logical underpinning.

Identity

Systems often need to compare two URLs to determine whether they refer to the same underlying object or not. For the purposes of this specification, the following rules apply:

- The query part of the URL (anything after `?`) is ignored
- The comparison of the document portion of the URL (i.e. not the server/port) is case sensitive
- The protocols `http:` and `https:` SHALL NOT be used to refer to different underlying objects
- If a port is specified, then the ports must be identical or the objects are different (due to the prevalence of port mapping and/or interface engines running on different ports). Ports should only be explicit when they have explicit meaning to the server

For example: `http://myserver.com/Patient/1` and `https://myserver.com/Patient/1` refer to the same underlying object, while `http://myserver.com:81/Patient/1` is a distinct entity from either of the above. This does not mean that the two addresses need to be treated the same, or that a server must serve both addresses, or that the content from the two addresses must be identical, but just that if these two addresses have the same identity, and if both are served, they must both represent the same underlying object. Systems are not required to check that this is true. Note: the identity comparison for protocols other than `http/https:` is undefined.

3.1.0.1.3 Resource Metadata and Versioning

Each resource has an associated set of [resource metadata elements \(resource.html#metadata\)](#). These map to the HTTP request and response using the following fields:

Metadata Item	Where found in HTTP
Logical Id (.id) (resource.html#id)	The Id is represented explicitly in the URL
Version Id (.meta.versionId) (resource.html#metadata)	The Version Id is represented in the <code>ETag</code> header
Last modified (.meta.lastUpdated) (resource.html#metadata)	HTTP Last-Modified header

Note that the Version Id is considered a "weak" ETag and `ETag` headers should be prefixed with `W/` and enclosed in quotes, for example:

```
ETag: W/"3141"
```

3.1.0.1.4 Security

Using HTTPS is optional, but all production exchange of healthcare data SHOULD use SSL and additional security as appropriate. See [HTTP Security \(security.html#http\)](#) for further information. Most operations will require user authentication, and all operations that do so are subject to [RBAC](#)  (https://en.wikipedia.org/wiki/Role-based_access_control) and/or [ABAC](#)  (https://en.wikipedia.org/wiki/Attribute-based_access_control), and some operations may depend on appropriate consent being granted.

Note: to support browser-based client applications, servers SHOULD implement [cross-origin resource sharing \(CORS\)](#)  (<http://enable-cors.org/>) for the interactions documented here.

See the [HTTP Security \(security.html#AccessDenied\)](#) guidance for further information about both security generally and the use of CORS.

3.1.0.1.5 HTTP Status Codes

This specification makes rules about the use of specific HTTP status codes in particular circumstances where the status codes SHALL map to particular states correctly, and only where the correct status code is not obvious. Other HTTP status codes may be used for other states as appropriate, and this particularly includes various authentication related status codes and redirects. Authentication redirects should not be interpreted to change the location of the resource itself (a common web programming error).

FHIR defines an [OperationOutcome resource \(operationoutcome.html\)](#) that can be used to convey specific detailed processable error information. For some combinations of interactions and specific return codes, an OperationOutcome is required to be returned as the content of the response. The OperationOutcome may be returned with any HTTP 4xx or 5xx response, but this is not required - many of these errors may be generated by generic server frameworks underlying a FHIR server.

3.1.0.1.6 Managing Return Content

If the server has a default timezone, it SHOULD return the default timezone in the HTTP Response headers using the " **Date** " header.

In the interests of managing band-width, this specification allows clients to specify what kind of content to return for resources.

3.1.0.1.7 conditional read

Clients may use the **If-Modified-Since** , or **If-None-Match** HTTP header on a **read** request. If so, they SHALL accept either a **304 Not Modified** as a valid status code on the response (which means that the content is unchanged since that date) or full content (either the content has changed, or the server does not support conditional request).

Servers can return **304 Not Modified** where content is unchanged because the **If-Modified-Since** date-time or the **If-None-Match** ETag was specified, or they can return the full content as normal. This optimisation is relevant in reducing bandwidth for caching purposes and servers are encouraged but not required to support this. If servers don't support conditional read, they just return the full content.

3.1.0.1.8 create/update/patch/transaction

These interactions are performed using **POST** , **PUT** or **PATCH** , and it may be appropriate for a server to return either only a status code, or also return the entire resource that is the outcome of the create or update (which may be different to that provided by the client). In the case of transactions this means returning a **Bundle** with just the **Bundle.entry.response** populated for each entry, and not the **Bundle.entry.resource** values.

The client can indicate whether the entire resource is returned using the **HTTP return preference** [↗\(https://tools.ietf.org/html/rfc7240#section-4.2\)](https://tools.ietf.org/html/rfc7240#section-4.2):

```
Prefer: return=minimal
Prefer: return=representation
Prefer: return=OperationOutcome
```

The first of these asks to return no body. The second asks to return the full resource. The third asks the server to return an **OperationOutcome** (operationoutcome.html) resource containing hints and warnings about the operation rather than the full resource. Servers SHOULD honor this header. In the absence of the header, servers may choose whether to return the full resource or not (but not the **OperationOutcome**; that should only be returned if explicitly requested). Note that this setting only applies to successful interactions. In case of failure, servers SHOULD always return a body that contains an **OperationOutcome** (operationoutcome.html) resource.

See also the **Asynchronous use pattern** (async.html) for another use of the **Prefer** header.

3.1.0.1.9 Content Types and encodings

The formal MIME-type for FHIR resources is **application/fhir+xml** or **application/fhir+json** . The correct mime type SHALL be used by clients and servers:

- XML: **application/fhir+xml**
- JSON: **application/fhir+json**
- RDF: **application/fhir+turtle** (only the Turtle format is supported)

Servers SHALL support server-driven content negotiation as described in **section 12** [↗\(https://tools.ietf.org/html/rfc7231#section-3.4\)](https://tools.ietf.org/html/rfc7231#section-3.4) of the HTTP specification.

Implementation Notes:

- The content type **application/x-www-form-urlencoded** (**Specification** [↗\(https://url.spec.whatwg.org/#application/x-www-form-urlencoded\)](https://url.spec.whatwg.org/#application/x-www-form-urlencoded)) is also accepted for posting **search** requests.
- If a client provides a generic mime type in the **Accept** header (**application/xml**, **text/json**, or **application/json**), the server SHOULD respond with the requested mime type, using the XML or JSON formats described in this specification as the best representation for the named mime type (except for binary - see the **note on the Binary resource** (binary.html#rest)).
- Note: between FHIR DSTU2 and STU3, the correct mime type was changed from **application/xml+fhir** and **application/json+fhir** to **application/fhir+xml** and **application/fhir+json** . Servers MAY also support the older mime types, and are encouraged to do so to smooth the transition process.
- **406 Not Acceptable** is the appropriate response when the **Accept** header requests a format that the server does not support, and **415 Unsupported Media Type** when the client posts a format that is not supported to the server.

UTF-8 encoding SHALL be used for the mime type `application/fhir` . This MAY be specified as a MIME type parameter to the `application/fhir` mime type, but is not required.

3.1.0.1.10 FHIR Version Parameter

This specification defines the MIME-type parameter `fhirVersion` as a parameter to indicate which version of the FHIR release a resource is based on:

Accept: `application/fhir+json; fhirVersion=4.0`

The value of this parameter is the [publication and major version number \(versions.html#versions\)](#) for the specification:

FHIR R1 (http://hl7.org/fhir/DSTU1) (DSTU 1)	0.0
FHIR R2 (http://hl7.org/fhir/DSTU2) (DSTU 2)	1.0
FHIR R3 (http://hl7.org/fhir/STU3) (STU3, or just R3)	3.0
FHIR R4 (http://hl7.org/fhir/r4) (this version)	4.0 (once published)

Intermediate balloted releases may also be encountered occasionally - see [publications directory](#) [\(<http://hl7.org/fhir/directory.html>\)](http://hl7.org/fhir/directory.html). Versions from before the publication of the [first DSTU](#) [\(<http://hl7.org/fhir/DSTU1/index.html>\)](http://hl7.org/fhir/DSTU1/index.html) (which is 0.0) are not supported.

This parameter can be used anywhere that the FHIR Mime type is used. When used in an HTTP request, the version parameter may be used on either the Content-Type header, or the Accept header, or both, and applies to the entire interaction (the behavior of the interactions as described on this page, the search parameters and functionality, and the accompanying conformance resources). It is an error for a client to attempt to use two different versions in the same interaction. For further information about specifying FHIR version, see [Managing FHIR Versions \(versioning.html\)](#).

3.1.0.1.11 General parameters

The following parameters are defined for use with all of the interactions defined on this page:

<code>_format</code>	Override the HTTP content negotiation - see immediately below
<code>_pretty</code>	Ask for a pretty printed response for human convenience - see below
<code>_summary</code>	Ask for a predefined short form of the resource in response - see Search Summary (search.html#summary)
<code>_elements</code>	Ask for a particular set of elements to be returned - see Search Elements (search.html#elements)

Note that the impact of `_elements` is not defined for interactions other than search where the response is a bundle that contains more than one type of resource.

`_format`

In order to support various implementation limitations, servers SHOULD support the optional `_format` parameter to specify alternative response formats by their MIME-types. This parameter allows a client to override the `accept` header value when it is unable to set it correctly due to internal limitations (e.g. XSLT usage). For the `_format` parameter, the values `xml`, `text/xml`, `application/xml`, and `application/fhir+xml` SHALL be interpreted to mean the [XML format \(xml.html\)](#), the codes `json`, `application/json` and `application/fhir+json` SHALL be interpreted to mean the [JSON format \(json.html\)](#), and the codes `turtle`, `application/fhir+turtle`, and `text/turtle` SHALL be interpreted to mean the [Turtle RDF format \(rdf.html\)](#). In addition, the values `html` and `text/html` are allowed.

Note: the `_format` parameter does not override the `Content-Type` header for the type of the body of a POST request. If neither the `accept` header nor the `_format` parameter are specified, the MIME-type of the content returned by the server is undefined and may vary.

`_pretty`

Clients that wish to request for pretty-printed resources (either in JSON or XML) can use the `_pretty` parameter:

GET `[base]/Patient/example?_pretty=true`

Value values are `true` and `false` . Since pretty printed or not makes no difference to the content, this is only of interest for development tools, and servers MAY choose to support this parameter.

3.1.0.1.12 Support for Versions

Servers that support this API SHOULD provide full version support - that is, populate and track `versionId` correctly, support `vread`, and implement [version aware updates](#). Supporting versions like this allows for related systems to track the correct version of information, and to keep integrity in clinical records. However, many current operational systems do not do this, and cannot easily be re-engineered to do so.

For this reason, servers are allowed to not provide versioning support and this API does not enforce that versioning is supported. Clients may elect to only interact with servers that do provide full versioning support. Systems declare their support for versioning in their [Capability Statements \(capabilitystatement-definitions.html#CapabilityStatement.rest.resource.versioning\)](#), where they can indicate one of three levels for versioning support:

- **no-version:** Versioning and `meta.version` is not supported (server) or used (client)
- **versioned:** Versioning and `meta.version` is supported (server) or used (client)
- **versioned-update:** Versioning and `meta.version` is supported, and version aware updates are used - Version ID must be correct for updates (server) or will be specified (If-match header) for updates (client)

Servers that do not support versioning SHALL ensure that `Resource.meta.versionId` is not present on resources they return, and SHALL update the value of `Resource.meta.lastUpdated` correctly.

3.1.0.2 read

The `read` interaction accesses the current contents of a resource. The interaction is performed by an HTTP `GET` command as shown:

```
GET [base]/[type]/[id] {?_format=[mime-type]}
```

This returns a single instance with the content specified for the resource type. This url may be accessed by a browser. The possible values for the [Logical Id \(resource.html#id\)](#) ("id") itself are described in the [id type \(datatypes.html#id\)](#). The returned resource SHALL have an `id` element with a value that is the `[id]`. Servers SHOULD return an `ETag` header with the versionId of the resource (if versioning is supported) and a `Last-Modified` header.

Note: Unknown resources and deleted resources are treated differently on a read: a `GET` for a deleted resource returns a `410` status code, whereas a `GET` for an unknown resource returns `404`. Systems that do not track deleted records will treat deleted records as an unknown resource. Since deleted resources may be brought back to life, servers MAY include an `ETag` on the error response when reading a deleted record to allow version contention management when a resource is brought back to life.

In addition, the search parameter `_summary` can be used when reading a resource:

```
GET [base]/[type]/[id] {?_summary=text}
```

This requests that only a subset of the resource content be returned, as specified in the `_summary` parameter, which can have the values `true`, `false`, `text`, `count` and `data` ([search.html#summary](#)). Note that a resource that only contains a subset of the data is not suitable for use as a base to update the resource, and might not be suitable for other uses. The same applies to the `_elements` ([search.html#elements](#)) parameter - both that it should be supported, and the subset implications. Servers SHOULD define a `Resource.meta.tag` with the [SUBSETTED \(v3/ObservationValue/cs.html#v3-ObservationValue-SUBSETTED\)](#) as a [Simple Tag \(resource.html#simple-tags\)](#) to explicitly mark such resources.

A HEAD request can also be used - [see below](#).

3.1.0.3 vread

The `vread` interaction performs a version specific read of the resource. The interaction is performed by an HTTP `GET` command as shown:

```
GET [base]/[type]/[id]/_history/[vid] {?_format=[mime-type]}
```

This returns a single instance with the content specified for the resource type for that version of the resource. The returned resource SHALL have an `id` element with a value that is the `[id]`, and a `meta.versionId` element with a value of `[vid]`. Servers SHOULD return an `ETag` header with the versionId (if versioning is supported) and a `Last-Modified` header.

The [Version Id \(resource.html#metadata\)](#) ("vid") is an opaque identifier that conforms to the same [format requirements \(datatypes.html#id\)](#) as a [Logical Id \(resource.html#id\)](#). The id may have been found by performing a history interaction (see below), by recording the version id from a content location returned from a `read` or from a version specific reference in a content model. If the version referred to is actually one where the resource was deleted, the server should return a `410` status code.

Servers are encouraged to support a version specific retrieval of the current version of the resource even if they do not provide access to previous versions. If a request is made for a previous version of a resource, and the server does not support accessing previous versions (either generally, or for this particular resource), it should return a **404 Not Found** error, with an operation outcome explaining that history is not supported for the underlying resource type or instance.

A HEAD request can also be used - [see below](#).

3.1.0.4 update

The **update** interaction creates a new current version for an existing resource or creates an initial version if no resource already exists for the given id. The **update** interaction is performed by an HTTP **PUT** command as shown:

```
PUT [base]/[type]/[id] {?_format=[mime-type]}
```

The request body SHALL be a [Resource \(resource.html\)](#) with an **id** element that has an identical value to the **[id]** in the URL. If no **id** element is provided, or the id disagrees with the id in the URL, the server SHALL respond with an HTTP **400** error code, and SHOULD provide an [OperationOutcome \(operationoutcome.html\)](#) identifying the issue. If the request body includes a [meta \(resource.html#meta\)](#), the server SHALL ignore the provided **versionId** and **lastUpdated** values. If the server supports versions, it SHALL populate the **meta.versionId** and **meta.lastUpdated** with the new correct values. Servers are allowed to review and alter the other metadata values, but SHOULD refrain from doing so (see [metadata description \(resource.html#meta\)](#) for further information). Note that there is no support for updating past versions - see notes on the [history](#) interaction.

A server SHOULD accept the resource as submitted when it accepts the update, and return the same content when it is subsequently read. However systems might not be able to do this; see the note on [transactional integrity](#) for discussion. Also, see [Variations between Submitted data and Retrieved data \(updates.html\)](#) for additional discussion around update behavior. Note that **update** generally updates the whole content of the resource. For partial updates, see [patch](#) below.

If the interaction is successful, the server SHALL return either a **200 OK** HTTP status code if the resource was updated, or a **201 Created** status code if the resource was created (or brought back to life/re-created), with a **Last-Modified** header, and an **ETag** header which contains the new **versionId** of the resource. If the resource was created (i.e. the interaction resulted in a **201 Created**), the server SHOULD return a **Location** header (this is for HTTP conformance; it's not otherwise needed). The body of response is as described in [Managing Return Content \(http.html#return\)](#).

Note: Servers MAY choose to preserve XML comments, instructions, and formatting or JSON whitespace when accepting updates, but are not required to do so. The impact of this on digital signatures may need to be considered.

3.1.0.4.1 Update as Create

Servers MAY choose to allow clients to **PUT** a resource to a location that does not yet exist on the server - effectively, allowing the client to define the id of the resource. Whether a server allows this is a deployment choice based on the nature of its relationships with the clients. While many servers will not allow clients to define their ids, there are several reasons why it may be necessary in some configurations:

- client is reproducing an existing data model on the server, and needs to keep original ids in order to retain ongoing integrity
- client is a server doing push based pub/sub (this is a special case of the first reason)
- multiple clients doing push in the context of agreed data model shared across multiple servers where ids are shared across servers

Alternatively, clients may be sharing an agreed identification model (e.g. key server, scoped identifiers, or UUIDs) where clashes do not arise. Note that this use of **update** has [security implications \(security.html#AccessDenied\)](#).

Servers can choose whether or not to support client defined ids, and indicate such to the clients using [CapabilityStatement.rest.resource.updateCreate \(capabilitystatement-definitions.html#CapabilityStatement.rest.resource.updateCreate\)](#).

3.1.0.4.2 Rejecting Updates

Servers are permitted to reject update interactions because of integrity concerns or other business rules, and return HTTP status codes accordingly (usually a **422**). Note that there are potential security issues relating to how rejections are handled. See the [security page \(security.html#AccessDenied\)](#) for more information.

Common HTTP Status codes returned on FHIR-related errors (in addition to normal HTTP errors related to security, header and content type negotiation issues):

- **400 Bad Request** - resource could not be parsed or failed basic FHIR validation rules (or multiple matches were found for conditional criteria)
- **401 Not Authorized** - authorization is required for the interaction that was attempted
- **404 Not Found** - resource type not supported, or not a FHIR end-point
- **405 Method Not allowed** - the resource did not exist prior to the update, and the server does not allow client defined ids

- **409 / 412** - version conflict management - see [below](#)
- **422 Unprocessable Entity** - the proposed resource violated applicable FHIR profiles or server business rules

Any of these errors SHOULD be accompanied by an [OperationOutcome](#) ([operationoutcome.html](#)) resource providing additional detail concerning the issue. In general, if an instance fails the constraints documented in the CapabilityStatement then the response should be a 400, whereas if the instance fails other non-externally described business rules, the response would be a 422 error. However, there's no expectation that servers will tightly adhere to this differentiation (nor is it clear that it makes much difference whether they do or not). In practice, servers may also return 5xx errors in these cases without being deemed non-conformant.

For additional information on how systems may behave when processing updates, refer to the [Variations between Submitted data and Retrieved data](#) ([updates.html](#)) page.

3.1.0.4.3 Conditional update

Unlike the rest of this page, the conditional create, update, patch and delete are trial use until further experience is gained with their use. Their status will be reviewed in a future version of FHIR.

The conditional update interaction allows a client to update an existing resource based on some identification criteria, rather than by [logical id](#) ([resource.html#meta](#)). To accomplish this, the client issues a PUT as shown:

```
PUT [base]/[type]?[search parameters]
```

When the server processes this update, it performs a search using its standard [search facilities](#) ([search.html](#)) for the resource type, with the goal of resolving a single logical id for this request. The action it takes depends on how many matches are found:

- **No matches, no id provided:** The server creates the resource.
- **No matches, id provided:** The server treats the interaction as an [Update as Create](#) interaction (or rejects it, if it does not support Update as Create)
- **One Match, no resource id provided OR (resource id provided and it matches the found resource):** The server performs the update against the matching resource
- **One Match, resource id provided but does not match resource found:** The server returns a 400 Bad Request error indicating the client id specification was a problem preferably with an [OperationOutcome](#)
- **Multiple matches:** The server returns a 412 Precondition Failed error indicating the client's criteria were not selective enough preferably with an [OperationOutcome](#)

This variant can be used to allow a stateless client (such as an interface engine) to submit updated results to a server, without having to remember the logical ids that the server has assigned. For example, a client updating the status of a lab result from "preliminary" to "final" might submit the finalized result using
PUT path/Observation?identifier=http://my-lab-system|123

Note that transactions and conditional create/update/delete are complex interactions and it is not expected that every server will implement them. Servers that don't support the conditional update SHOULD return an HTTP 400 error and MAY include an [OperationOutcome](#) ([operationoutcome.html](#)).

The resource MAY contain an `id` element, but does not need to (this is one of the few cases where a resource exists without an `id` element). If an `id` is provided, the server SHALL ignore it - see documentation for the [update](#) interaction.

3.1.0.5 Managing Resource Contention

[Lost Updates](#) [\(http://www.w3.org/1999/04/Editing/\)](http://www.w3.org/1999/04/Editing/), where two clients update the same resource, and the second overwrites the updates of the first, can be prevented using a combination of the [ETag](#) [\(<https://tools.ietf.org/html/rfc7232#section-2.3>\)](https://tools.ietf.org/html/rfc7232#section-2.3) and [If-Match](#) [\(<https://tools.ietf.org/html/rfc7232#section-3.1>\)](https://tools.ietf.org/html/rfc7232#section-3.1) header. This is also known as 'Optimistic Locking'.

To support this usage, servers SHOULD always return an `ETag` header with each resource:

```
HTTP 200 OK
Date: Sat, 09 Feb 2013 16:09:50 GMT
Last-Modified: Sat, 02 Feb 2013 12:02:47 GMT
ETag: W/"23"
Content-Type: application/fhir+json
```

If provided, the value of the **ETag** SHALL match the value of the version id for the resource. Servers are allowed to generate the version id in whatever fashion that they wish, so long as they are valid according to the [id \(dataatypes.html#id\)](#) data type, and are unique within the address space of all versions of the same resource. When resources are returned as part of a bundle, there is no **ETag**, and the **versionId** of the resource is used directly.

If the client wishes to request a version aware update, it submits the request with an **If-Match** header that quotes the ETag from the server:

```
PUT /Patient/347 HTTP/1.1
If-Match: W/"23"
```

If the version id given in the **If-Match** header does not match, the server returns a **412 Precondition Failed** status code instead of updating the resource.

Servers can require that clients provide an **If-Match** header by returning **400 Client Error** status codes when no **If-Match** header is found. Note that a **409 Conflict** can be returned when the server detects the update cannot be done (e.g. due to server side pessimistic locking).

3.1.0.6 patch

As an alternative to updating an entire resource, clients can perform a patch operation. This can be useful when a client is seeking to minimize its bandwidth utilization, or in scenarios where a client has only partial access or support for a resource. The **patch** interaction is performed by an HTTP **PATCH** command as shown:

```
PATCH [base]/[type]/[id] {?_format=[mime-type]}
```

The body of a PATCH operation SHALL be either:

- a [JSON Patch](https://tools.ietf.org/html/rfc6902) document with a content type of **application/json-patch+json**
- an [XML Patch](http://tools.ietf.org/html/rfc5261) document with a content type of **application/xml-patch+xml**
- a [FHIRPath Patch](#) parameters resource with [FHIR Content Type](#)

In either case, the server SHALL process its own copy of the resource in the format indicated, applying the operations specified in the document, following the relevant PATCH specification. When the operations have all been processed, the server processes the resulting document as an **Update** operation; all the version and error handling etc. apply as specified, as does the [Prefer Header](#).

Processing PATCH operations may be very version sensitive. For this reason, servers SHALL support conditional PATCH, which works exactly the same as specified for update in [Concurrency Management](#). Clients SHOULD always consider using version specific PATCH operations so that inappropriate actions are not executed.

Unlike this rest of this page, the conditional create, update, patch and delete are trial use until further experience is gained with their use. Their status will be reviewed in a future version of FHIR.

In addition, servers that support PATCH, and that support [Conditional Update](#) SHOULD also support conditional PATCH. When the server processes a conditional PATCH, it performs a search using its standard [search facilities](#) for the resource type, with the goal of resolving a single logical id for this request. The action it takes depends on how many matches are found:

- **No matches:** The server returns a 404 Not Found
- **One Match:** The server performs the update against the matching resource
- **Multiple matches:** The server returns a **412 Precondition Failed** error indicating the client's criteria were not selective enough

The server SHALL ensure that the narrative in a resource is not clinically unsafe after the PATCH operation is performed. Exactly how this is defined and can be achieved depends on the context, and how narrative is being maintained, but servers may wish to consider:

- If the existing narrative has a status != **generated**, the server could reject the PATCH operation
- The server could regenerate the narrative once the operation has been applied to the data
- In some limited circumstances, an XML PATCH operation could update the narrative
- The server could delete the narrative, on the basis that some later process will be able to populate it correctly

Processing XML Patch documents is tricky because of namespace handling. Servers SHALL handle namespaces correctly, but note that FHIR resources only contain two XML namespaces, for FHIR (<http://hl7.org/fhir>) and XHTML (<http://www.w3.org/1999/xhtml>).

For PATCH Examples, see [the FHIR test cases \(test-cases.zip\)](#).

Patch operations may be performed as part of Batch or Transaction Operations using the FHIRPath Patch format.

Patch is not defined for all resources - see [not about PATCH on Binary \(binary.html#patch\)](#).

3.1.0.7 delete

The **delete** interaction removes an existing resource. The interaction is performed by an HTTP **DELETE** command as shown:

```
DELETE [base]/[type]/[id]
```

The request body SHALL be empty.

A delete interaction means that subsequent [non-version specific reads](#) of a resource return a **410** HTTP status code and that the resource is no longer found through [search](#) interactions. Upon successful deletion, or if the resource does not exist at all, the server should return either a **200 OK** if the response contains a payload, or a **204 No Content** with no response payload, or a **202 Accepted** if the server wishes to be non-committal about the outcome of the delete.

Whether to support delete at all, or for a particular resource type or a particular instance is at the discretion of the server based on the policy and business rules that apply in its context. If the server refuses to delete resources of that type as a blanket policy, then it should return the **405** Method not allowed status code. If the server refuses to delete a resource because of reasons specific to that resource, such as referential integrity, it should return the **409** Conflict status code. Note that the servers MAY choose to enforce business rules regarding deletion of resources that are being referenced by other resources, but they also MAY NOT do so. Performing this interaction on a resource that is already deleted has no effect, and the server should return either a **200 OK** if the response contains a payload, or a **204 No Content** with no response payload. Resources that have been deleted may be "brought back to life" by a subsequent [update](#) interaction using an HTTP **PUT**.

Many resources have a status element that overlaps with the idea of deletion. Each resource type defines what the semantics of the deletion interactions are. If no documentation is provided, the deletion interaction should be understood as deleting the record of the resource, with nothing about the state of the real-world corresponding resource implied.

For servers that maintain a version history, the **delete** interaction does not remove a resource's version history. From a version history respect, deleting a resource is the equivalent of creating a special kind of history entry that has no content and is marked as deleted. Note that there is no support for deleting past versions - see notes on the [history](#) interaction. Since deleted resources may be brought back to life, servers MAY include an **Etag** on the delete response to allow version contention management when a resource is brought back to life.

Note that irrespective of this rule, servers are free to completely delete the resource and it's history if policy or business rules make this the appropriate action to take.

3.1.0.7.1 Conditional delete

Unlike this rest of this page, the conditional create, update, patch and delete are trial use until further experience is gained with their use. Their status will be reviewed in a future version of FHIR.

The conditional delete interaction allows a client to delete an existing resource based on some selection criteria, rather than by a specific [logical id \(resource.html#meta\)](#). To accomplish this, the client issues an HTTP **DELETE** as shown:

```
DELETE [base]/[type]/?[search parameters]
```

When the server processes this delete, it performs a search as specified using the standard [search facilities \(search.html\)](#) for the resource type. The action it takes depends on how many matches are found:

- **No matches** or **One Match**: The server performs an ordinary **delete** on the matching resource
- **Multiple matches**: A server may choose to delete all the matching resources, or it may choose to return a **412** Precondition Failed error indicating the client's criteria were not selective enough. A server indicates whether it can delete multiple resources in its [Capability Statement \(.rest.resource.conditionalDelete\)](#) ([capabilitystatement-definitions.html#CapabilityStatement.rest.resource.conditionalDelete](#)).

This variant can be used to allow a stateless client (such as an interface engine) to delete a resource on a server, without having to remember the logical ids that the server has assigned. For example, a client deleting a lab atomic result might delete the resource using

```
DELETE /Observation?identifier=http://my-lab-system|123 .
```

Note that transactions and conditional create/update/delete are complex interactions and it is not expected that every server will implement them. Servers that don't support the conditional update SHOULD return an HTTP 400 error and MAY include an [OperationOutcome \(operationoutcome.html\)](#).

3.1.0.8 create

The **create** interaction creates a new resource in a server-assigned location. If the client wishes to have control over the id of a newly submitted resource, it should use the **update** interaction instead. The **create** interaction is performed by an HTTP POST command as shown:

```
POST [base]/[type] {?_format=[mime-type]}
```

The request body SHALL be a FHIR Resource. The resource does not need to have an **id** element (this is one of the few cases where a resource exists without an **id** element). If an **id** is provided, the server SHALL ignore it. If the request body includes a **meta** ([resource.html#meta](#)), the server SHALL ignore the existing **versionId** and **lastUpdated** values. The server SHALL populate the **id**, **meta.versionId** and **meta.lastUpdated** with the new correct values. Servers are allowed to review and alter the other metadata values, but SHOULD refrain from doing so (see [metadata description \(resource.html#meta\)](#) for further information).

A server SHOULD otherwise accept the resource as submitted when it accepts the create, and return the same content when it is subsequently read. However some systems might not be able to do this; see the note on [transactional integrity](#) for discussion.

The server returns a 201 Created HTTP status code, and SHALL also return a **Location** header which contains the new [Logical Id \(resource.html#metadata\)](#) and [Version Id \(resource.html#metadata\)](#) of the created resource version:

```
Location: [base]/[type]/[id]/_history/[vid]
```

where **[id]** and **[vid]** are the newly created id and version id for the resource version. The Location header should be as specific as possible - if the server understands versioning, the version is included. If a server does not track versions, the Location header will just contain **[base]/[type]/[id]**. The Location MAY be an absolute or relative URL.

Servers SHOULD return an **ETag** header with the **versionId** (if versioning is supported) and a **Last-Modified** header. The body of response is as described in [Managing Return Content \(http.html#return\)](#).

When the resource syntax or data is incorrect or invalid, and cannot be used to create a new resource, the server returns a 400 Bad Request HTTP status code. When the server rejects the content of the resource because of business rules, the server returns a 422 Unprocessable Entity error HTTP status code. In either case, the server SHOULD include a response body containing an [OperationOutcome \(operationoutcome.html\)](#) with detailed error messages describing the reason for the error.

Note: Servers MAY choose to preserve XML comments, instructions, and formatting or JSON whitespace when accepting creates, but are not required to do so. The impact of this on digital signatures may need to be considered.

Common HTTP Status codes returned on FHIR-related errors (in addition to normal HTTP errors related to security, header and content type negotiation issues):

- **400 Bad Request** - resource could not be parsed or failed basic FHIR validation rules
- **404 Not Found** - resource type not supported, or not a FHIR end-point
- **422 Unprocessable Entity** - the proposed resource violated applicable FHIR profiles or server business rules. This should be accompanied by an [OperationOutcome \(operationoutcome.html\)](#) resource providing additional detail

In general, if an instance fails the constraints documented in the CapabilityStatement then the response should be a 400, whereas if the instance fails other non-externally described business rules, the response would be a 422 error. However, there's no expectation that servers will tightly adhere to this differentiation (nor is it clear that it makes much difference whether they do or not). In practice, servers may also return 5xx errors in these cases without being deemed non-conformant.

For additional information on how systems may behave when processing updates, refer to the [Variations between Submitted data and Retrieved data \(updates.html\)](#) page.

3.1.0.8.1 Conditional create

Unlike this rest of this page, the conditional create, update, patch and delete are trial use until further experience is gained with their use. Their status will be reviewed in a future version of FHIR.

The conditional **create** interaction allows a client to create a new resource only if some equivalent resource does not already exist on the server. The client defines what equivalence means in this case by supplying a FHIR search query using an HL7 defined extension header " **If-None-Exist** " as shown:

```
If-None-Exist: [search parameters]
```

The parameter just contains the search parameters (what would be in the URL following the "?").

When the server processes this create, it performs a search as specified using its standard [search facilities \(search.html\)](#) for the resource type. The action it takes depends on how many matches are found:

- **No matches:** The server processes the create as above
- **One Match:** The server ignores the post and returns `200 OK`
- **Multiple matches:** The server returns a `412 Precondition Failed` error indicating the client's criteria were not selective enough

This variant can be used to avoid the risk of two clients creating duplicate resources for the same record. For example, a client posting a new lab result might specify `If-None-Exist: identifier=http://my-lab-system|123` to ensure it does not create a duplicate record.

Note that transactions and conditional create/update/delete are complex interactions and it is not expected that every server will implement them. Servers that don't support the conditional update SHOULD return an HTTP `400` error and MAY include an [OperationOutcome \(operationoutcome.html\)](#).

3.1.0.9 search

This interaction searches a set of resources based on some filter criteria. The interaction can be performed by several different HTTP commands.

```
GET [base]/[type]?[parameters]&_format=[mime-type]}
```

This searches all resources of a particular type using the criteria represented in the parameters.

Because of the way that some user agents and proxies treat `GET` and `POST` requests, in addition to the get based search method above, servers that support *search* SHALL also support a `POST` based search:

```
POST [base]/[type]/_search?[parameters]&_format=[mime-type]}
Content-Type: application/x-www-form-urlencoded

param1=value&param2=value
```

This has exactly the same semantics as the equivalent `GET` command. Note that in the `POST` variant, parameters may appear in both the URL and the body. Parameters have the same meaning in either place. Since parameters can repeat, putting them in both places is the same as repeating them (which is valid for some parameters and not for others).

Note: Supporting `GET` means that PHI (Personal health information) might appear in search parameters, and therefore in HTTP logs. For this reason logs should be regarded as being as sensitive as the resources themselves. This is a general requirement irrespective of the use of `GET` - see the [security page \(security.html#audit\)](#) for further commentary.

All these search interactions take a series of parameters that are a series of `name=value` pairs encoded in the URL (or as an `application/x-www-form-urlencoded` ([Specification](https://url.spec.whatwg.org/#application/x-www-form-urlencoded) <https://url.spec.whatwg.org/#application/x-www-form-urlencoded>)) submission for a `POST`). (See [W3C HTML forms](http://www.w3.org/TR/REC-html40/interact/forms.html#form-content-type) <http://www.w3.org/TR/REC-html40/interact/forms.html#form-content-type>).

Note: `application/x-www-form-urlencoded` is supported for `POST` so that invoking a search by `GET` or `POST` can be done from HTML forms in a browser (though considerable active content might be required in the browser), although this is not the main usage.

A `HEAD` request can also be used - [see below](#).

Searches are processed as specified for the [Search handling mechanism \(search.html\)](#).

If the search succeeds, the server SHALL return a `200 OK` HTTP status code and the return content SHALL be a [Bundle \(bundle.html\)](#) with `type (bundle-definitions.html#Bundle.type) = searchset` containing the results of the search as a collection of zero or more resources in a defined order. Note that resources returned in the search bundle MAY be located on the another server than the one that performs the search (i.e. the [Bundle.entry.fullUrl \(bundle-definitions.html#Bundle.entry.fullUrl\)](#) may

be different to the [base] from the search URL).

The result collection can be long, so servers may use paging. If they do, they SHALL use the method [described below](#) (adapted from [RFC 5005 \(Feed Paging and Archiving\)](#) (<https://tools.ietf.org/html/rfc5005>)) for breaking the collection into pages if appropriate. The server MAY also return an [OperationOutcome](#) ([operationoutcome.html](#)) resource within the `searchset` Bundle entries that contains additional information about the search; if one is sent it SHALL NOT include any issues with a `fatal` or `error` severity ([valueset-issue-severity.html](#)), and it SHALL be marked with a [Bundle.entry.search.mode](#) ([valueset-search-entry-mode.html](#)) of `outcome` .

If the search fails (cannot be executed, not that there are no matches), the return value SHALL be a status code 4xx or 5xx with an [OperationOutcome](#) ([operationoutcome.html](#)).

Common HTTP Status codes returned on FHIR-related errors (in addition to normal HTTP errors related to security, header and content type negotiation issues):

- **400 Bad Request** - search could not be processed or failed basic FHIR validation rules
- **401 Not Authorized** - authorization is required for the interaction that was attempted
- **404 Not Found** - resource type not supported, or not a FHIR end-point

3.1.0.9.1 Variant Searches

To search a [compartment](#) ([compartmentdefinition.html](#)), for either all possible resources or for a particular resource type, respectively:

```
GET [base]/[Compartment]/[id]/*{?[parameters]}&_format=[mime-type]}
GET [base]/[Compartment]/[id]/[type]{?[parameters]}&_format=[mime-type]}
```

In the first URL the character " * " appears in the URL as a literal to mean 'all types'. So, for example, to retrieve all the observation resources for a particular LOINC code associated with a particular encounter:

```
GET [base]/Encounter/23423445/Observation?code=2951-2  &_format=[mime-type]}
```

Note that there are specific operations defined to support fetching [an entire patient record](#) ([patient-operation-everything.html](#)) or [all record for an encounter](#) ([encounter-operation-everything.html](#)).

It is also possible to search across multiple resource types:

```
GET [base]?_type=Condition,Observation&[parameters]&_format=[mime-type]}
```

This is a request to search on both Condition and Observation. In this case, the only parameters that can be used are those defined for both Condition and Observation (using [SearchParameter.base](#) ([searchparameter-definitions.html#SearchParameter.base](#)) - see [Cross-resource Search Parameters](#) ([searchparameter-registry.html#common](#))), or the [parameters defined for all resources](#) ([resource.html#search](#)). If a search lists types not listed in `SearchParameter.base` for any of the parameters, this is an error, and a server SHOULD return a 400 status. It is also possible to search on all types at once:

```
GET [base]?[parameters]&_format=[mime-type]}
```

When searching all resources at once, the only search parameters that be can be used in global search like this are the [base parameters that apply to all resources](#) ([resource.html#search](#)).

3.1.0.10 capabilities

The `capabilities` interaction retrieves the information about a server's capabilities - which portions of this specification it supports. The interaction is performed by an HTTP `GET` command as shown:

```
GET [base]/metadata{?mode=[mode]} &_format=[mime-type]}
```

Applications SHALL return a resource that describes the functionality of the server end-point. The information returned depends on the value of the `mode` parameter:

<code>full</code> (or mode not present)	A Capability Statement (capabilitystatement.html) that specifies which resource types and interactions are supported
---	--

normative	As above, but only the normative portions of the Capability Statement
terminology	A TerminologyCapabilities (terminologycapabilities.html) resource that provides further information about terminologies which are supported by the server

Servers MAY ignore the mode parameter and return a CapabilityStatement resource. Note: servers might be required to support this parameter in further versions of this specification.

If a 404 Unknown is returned from the GET , FHIR (or the specified version) is not supported on the nominated service url. An ETag header SHOULD be returned with the Response. The value of the ETag header SHALL change if the returned resource changes.

Servers SHOULD check for the [fhirVersion MIME-type parameter](#) when processing this request.

The resource returned typically has an arbitrary id, and no meta element, though it is not prohibited. Capability statements can become quite large; servers are encouraged to support the [summary \(search.html#summary\)](#) and [elements \(search.html#elements\)](#) parameters on the capabilities interaction, though this is not required. In addition, servers are encouraged to implement the [\\$subset \(capabilitystatement-operation-subset.html\)](#) and [\\$implements \(capabilitystatement-operation-implements.html\)](#) operations to make it easy for a client to check conformance.

In addition to this capabilities interaction, a server may also choose to provide the standard set of interactions (read , search , create , update) defined on this page for the [CapabilityStatement Resource \(capabilitystatement.html\)](#) end-point. This is different from the capabilities interaction:

capabilities interaction	returns a capability statement describing the server's current operational functionality
CapabilityStatement end-point	manages a repository of capability statements (e.g. the HL7 capability statement registry)

All servers are required to support the capabilities interaction, but servers may choose whether they wish to support the CapabilityStatement end-point, just like any other end-point.

Implementation Note: In DSTU 2 and earlier, the resource that this interaction returned was named "Conformance". Clients often connect to a server, and use the capabilities interaction to check whether they are version and/or feature compatible with the server. Such clients should be able to process either a Conformance or a CapabilityStatement resource.

3.1.0.11 batch/transaction

The batch and transaction interactions submit a set of actions to perform on a server in a single HTTP request/response. The actions may be performed independently as a "batch", or as a single atomic "transaction" where the entire set of changes succeed or fail as a single entity. Multiple actions on multiple resources of the same or different types may be submitted, and they may be a mix of other interactions defined on this page (e.g. read , search , create , update , delete , etc.), or using the [operations \(operations.html\)](#) framework.

The transaction mode is especially useful where one would otherwise need multiple interactions, possibly with a risk of loss of referential integrity if a later interaction fails (e.g. when storing a Provenance resource and its corresponding target resource, or and [IHE-MHD transaction "Provide Document Resources"](#) (https://wiki.ihe.net/index.php/MHD_Implementation_Guide) with a DocumentManifest, and some number of DocumentReference, List, and Binary resources).

Note that transactions and conditional create/update/delete are complex interactions and it is not expected that every server will implement them. Servers that don't support the batches or transactions SHOULD return an HTTP 400 error and MAY include an [OperationOutcome \(operationoutcome.html\)](#).

A batch or transaction interaction is performed by an HTTP POST command as shown:

```
POST [base] {?_format=[mime-type]}
```

The content of the post submission is a [Bundle \(bundle.html\)](#) with [Bundle.type \(bundle-definitions.html#Bundle.type\)](#) = [batch \(valueset-bundle-type.html#batch\)](#) or [transaction \(valueset-bundle-type.html#transaction\)](#) . Each entry SHALL carry request details ([Bundle.entry.request \(bundle-definitions.html#Bundle.entry.request\)](#)) that provides the HTTP details of the action in order to inform the system processing the batch or transaction what to do for the entry. If the HTTP command is a PUT or POST , then the entry SHALL contain a resource for the body of the action. The resources in the bundle are each processed separately as if they were an individual interaction or operation as otherwise described on this page, or the [Operations framework \(operations.html\)](#). The actions are subject to the normal processing for each, including the [meta element \(resource.html#meta\)](#), verification and version aware updates, and [transactional integrity](#). In the case of a batch each entry is treated as if an individual interaction or operation, in the case of a transaction all interactions or operations either succeed or fail together (see below).

Examples:

- [Transaction Example \(bundle-transaction.html\)](#) with [Matching Response \(bundle-response.html\)](#).
- [Batch request to fetch Meds & Allergies \(bundle-request-medsallergies.html\)](#) with [Response \(bundle-response-medsallergies.html\)](#).
- [Batch request to fetch simple Patient Summary \(bundle-request-simplesummary.html\)](#) with [Response \(bundle-response-simplesummary.html\)](#).

3.1.0.11.1 Batch Processing Rules

For a `batch`, there SHALL be no interdependencies between the different entries in the Bundle that cause change on the server. The success or failure of one change SHOULD not alter the success or failure or resulting content of another change. Servers SHOULD validate that this is the case. Note that it is considered that servers execute the batch in the same order as that specified below for transactions, though the order of execution should not matter given the previous rule.

References within a `Bundle.entry.resource` to another `Bundle.entry.resource` that is being created within the batch are considered to be non-conformant.

When processing the batch, the HTTP response code is 200 Ok if the batch was processed correctly, regardless of the success of the operations within the Batch. To determine the status of the operations, look inside the returned Bundle. A response code on an entry of other than 2xx (200, 202 etc) indicates that processing the request in the entry failed.

3.1.0.11.2 Transaction Processing Rules

For a `transaction`, servers SHALL either accept all actions and return a `200 OK`, along with a response bundle (see below), or reject all resources and return an HTTP `400` or `500` type response. It is not an error if the submitted bundle has no resources in it. The outcome of processing the transaction SHALL NOT depend on the order of the resources in the transaction. A resource can only appear in a transaction once (by identity).

Because of the rules that a transaction is atomic where all actions pass or fail together and the order of the entries doesn't matter, there is a particular order in which to process the actions:

1. Process any `DELETE` interactions
2. Process any `POST` interactions
3. Process any `PUT` or `PATCH` interactions
4. Process any `GET` or `HEAD` interactions
5. Resolve any conditional references

If any resource identities (including resolved identities from conditional update/delete) overlap in steps 1-3, then the transaction SHALL fail.

A transaction may include references from one resource to another in the bundle, including circular references where resources refer to each other. When the server assigns a new id to any resource in the bundle which has a POST method as part of the processing rules above, it SHALL also update any references to that resource in the same bundle as they are processed (see [about Ids in a bundle \(bundle.html#bundle-unique\)](#)). References to resources that are not part of the bundle are left untouched. Version-specific references should remain as version-specific references after the references have been updated. Note that when building a transaction, a client can use arbitrarily chosen version references since they will all be re-assigned anyway. Servers SHALL replace all matching links in the bundle, whether they are found in the resource ids, [resource references \(references.html\)](#), elements of type `uri (datatypes.html#uri)`, `url (datatypes.html#url)`, `oid (datatypes.html#oid)`, `uuid (datatypes.html#uuid)`, and `<a href="" & <img src=""` in the narrative. Elements of type `canonical (datatypes.html#canonical)` are not replaced.

When processing a "POST" (create), the full URL is treated as the id of the resource on the source, and is ignored; the server generates an id for the resource. For updates, the server performs a mapping between the fullUrl specified and the local URL the server knows that instance as, if possible. If the server does not have a mapping for the fullUrl, the server ignores the base URL and attempts an update assuming the base is the same as the server base. This allows the same transaction bundle to be sent to multiple systems without changing the fullUrls for each target.

When processing a batch or transaction, a server MAY choose to honor existing logical ids (e.g. `Observation/1234` remains as `Observation/1234` on the server), but since this is only [safe in controlled circumstances](#), servers may choose to assign new ids to all submitted resources, irrespective of any claimed logical `id` in the resource, or `fullUrl` on entries in the batch/transaction.

Conditional References

When constructing the bundle, the client might not know the logical id of a resource, but it may know identifying information - e.g. an identifier. This situation arises commonly when building transactions from v2 messages. The client could resolve that identifier to a logical id using a search, but that would mean that the resolution to a logical id does not occur within the same transaction as the commit (as well as significantly complicating the client). Because of this, in a transaction (and only in a transaction), references to resources may be replaced by a search URI that describes how to find the correct reference:

```

<Bundle xmlns="http://hl7.org/fhir">
  <id value="20160113160203" />
  <type value="transaction" />
  <entry>
    <fullUrl value="urn:uuid:c72aa430-2ddc-456e-7a09-dea8264671d8" />
    <resource>
      <Observation>
        <subject>
          <reference value="Patient?identifier=12345"/>
        </subject>
        <!--! rest of resource omitted -->
      </Observation>
    </resource>
  </entry>
  <request>
    <method value="POST" />
  </request>
</Bundle>

```

The search URI is relative to the server's [base] path, and always starts with a resource type: `[type]?parameters...`. Only filtering parameters are allowed; none of the parameters that control the return of resources are relevant.

When processing transactions, servers SHALL:

- check all references for search URIs
- For search URIs, use the search to locate matching resources
- if there are no matches, or multiple matches, the transaction fails, and an error is returned to the user
- if there is a single match, the server replaces the search URI with a reference to the matching resource

3.1.0.11.3 Batch/Transaction Response

For a batch, or a successful transaction, the response the server SHALL return a [Bundle \(bundle.html\)](#) with `type (bundle-definitions.html#Bundle.type)` set to `batch-response` or `transaction-response` that contains one entry for each entry in the request, in the same order, with the outcome of processing the entry. For a failed transaction, the server returns a single [OperationOutcome \(operationoutcome.html\)](#) instead of a Bundle.

A client may use the returned Bundle to track the outcomes of processing the entry, and the identities assigned to the resources by the server. Each entry element SHALL contain a `response` element which details the outcome of processing the entry - the HTTP status code, and the location and `Etag` header values, which are used for identifying and versioning the resources. In addition, a resource may be included in the entry, as specified by the [Prefer](#) header.

3.1.0.11.4 Accepting other Bundle types

A server may choose to accept bundle types other than `batch` or `transaction` when `POST` ed to the [base] URL.

Bundles of type `history` inherently have the same structure as a `transaction`, and can be treated as either a transaction or batch, so servers SHOULD accept a history Bundle - this makes it possible to replicate data from one server to another easily using a pub/sub model. Note, however, that the original transaction boundaries might not be represented in a history list, and a resource may occur more than once in a history list, so servers processing history bundles must have some strategy to manage this. When processing a history bundle via a transaction, any entries with the request method of `POST` must use the `Bundle.entry.resource.id` (which must match the `Bundle.entry.response.location`) for that resource so that references are preserved.

For other Bundle types, should the server choose to accept them, there will be no `request` element (note that every entry will have a resource). In this case, the server treats the entry as either a create or an update interaction, depending on whether it recognizes the identity of the resource - if the identity of the resource refers to a valid location on the server, it should treat it as an update to that location. Note: this option allows a client to delegate the matching process to the server.

3.1.0.12 history

The history interaction retrieves the history of either a particular resource, all resources of a given type, or all resources supported by the system. These three variations of the history interaction are performed by HTTP `GET` command as shown:

```
GET [base]/[type]/[id]/_history{?[parameters]&_format=[mime-type]}
GET [base]/[type]/_history{?[parameters]&_format=[mime-type]}
GET [base]/_history{?[parameters]&_format=[mime-type]}
```

The return content is a [Bundle \(bundle.html\)](#) with `type (bundle-definitions.html#Bundle.type)` set to `history` containing the specified version history, sorted with oldest versions last, and including deleted resources. Each entry SHALL minimally contain at least one of: a `resource` which holds the resource as it is at the conclusion of the interaction, or a `request` with `entry.request.method`. The `request` provides information about the result of the interaction that led to this new version, and allows, for instance, a subscriber system to differentiate between newly created resources and updates to existing resources. The principal reason a `resource` might be missing is that the resource was changed by some other channel rather than via the RESTful interface. If the `entry.request.method` is a `PUT` or a `POST`, the entry SHALL contain a resource.

The interactions `create`, `update`, and `delete` create history entries. Other interactions do not (note that these operations may produce side-effects such as new `AuditEvent` resources; these are represented as create interactions in their own right). New resources or updates to existing resources that are triggered by operations also appear in the history, as do updates to the resources that result from interactions outside the scope of the RESTful interface.

A HEAD request can also be used - [see below](#).

A `create` interaction is represented in a history interaction in the following way:

```
<entry>
  <resource>
    <Patient>
      <!-- the id of the created resource -->
      <id value="23424"/>
      <!-- snip -->
    </Patient>
  </resource>
  <request>
    <!-- POST: this was a create -->
    <method value="POST"/>
    <url value="Patient"/>
  </request>
  <!-- response carries the instant the server processed the create -->
  <response>
    <lastModified value="2014-08-15T10:35:02.034Z"/>
  </response>
</entry>
```

A `delete` interaction is represented in a history interaction in the following way:

```
<entry>
  <!-- no resource included for a delete -->
  <request>
    <method value="DELETE"/>
    <url value="Patient/[id]"/>
  </request>
  <!-- response carries the instant the server processed the delete -->
  <response>
    <lastModified value="2014-08-20T11:05:34.174Z"/>
  </response>
</entry>
```

Notes:

- conditional creates, updates and deletes are converted to direct updates and deletes in a history list
- operations do not appear directly in the history log, but side effects (e.g. creation of audit logs, stored binaries, etc.) will appear where relevant
- The resource in the entry is the resource as processed by the server, not as submitted by the client ([may be different \(updates.html\)](#))

- The server SHOULD populate at least [response.lastModified \(bundle-definitions.html#Bundle.entry.response.lastModified\)](#) so the time of processing is clear in the history record
- Servers may choose to only record successful interactions. Servers may choose to only use 200 OK instead of other more specific success codes
- There may be more than one version of a given resource in the history

In addition to the standard `_format` parameter, the parameters to this interaction may also include:

<code>_count</code> : integer (search.html#number)	The maximum number of search results on a page, excluding related resources included by <code>_include</code> or <code>_revinclude</code> or <code>OperationOutcomes</code> . The server is not bound to return the number requested, but cannot return more
<code>_since</code> : instant (search.html#date)	Only include resource versions that were created at or after the given instant in time
<code>_at</code> : date(Time) (search.html#date)	Only include resource versions that were current at some point during the time period specified in the date time value (see Search notes on date searching (search.html#date))
<code>_list</code> : reference (search.html#reference)	Only include resource versions that are referenced in the specified list (current list references (lifecycle.html#current) are allowed)

Each of these parameters SHALL NOT appear more than once.

The history list can be restricted to a limited period by specifying a `_since` parameter which contains a full date time with time zone. Clients should be aware that due to timing imprecision, they may receive notifications of a resource update on the boundary instant more than once. Servers are not required to support a precision finer than by second.

The updates list can be long, so servers may use paging. If they do, they SHALL use the method [described below](#) for breaking the list into pages if appropriate, and respect the specified `_count` across pages.

The `history` interaction can be used to set up a subscription from one system to another, so that resources are synchronized between them. Refer to the [Subscription resource \(subscription.html\)](#) for an alternate means of system synchronization.

Additional Notes about maintaining a history of resources:

- The history is a record version history on a per-resource basis. It is not intended to support concurrent versions, or multi-branch version history
- Accordingly, there is no way to update or delete past versions of the record, except that the [metadata can be modified \(resource-operations.html\)](#) (mainly for access control purposes)
- All past versions of a resource are considered to be superceded, and no longer active, but retained for audit/integrity purposes
- In the case that a past version of a resource needs to be explicitly documented as 'entered-in-error' ([lifecycle.html#error](#)), use a [Provenance \(provenance.html\)](#) resource pointing to the past version of the resource
- When tracing the history of a specific resource, applications should retrieve any provenance resources relating to the resource or its past versions
- If a request is made for a history that is not available (e.g. the system does not keep a history for the type, or the particular instance), the server should return a 404 Not Found along with an [OperationOutcome \(operationoutcome.html\)](#) explaining the problem

There is a caveat with the `_list` parameter, associated with changes to the list while making repeated periodic queries; if the list changes, the response will include changes to the resources in the list for the period specified, but will omit both later changes to items no longer in the list, or older changes associated with items in the list. This might not be a problem, but implementers should be aware of this issue.

3.1.0.13 Transactional Integrity

When processing [create](#) and [update](#) interactions, a FHIR server is not obliged to accept the entire resource as it is; when the resource is retrieved through a [read](#) interaction subsequently, the resource may be different. The difference may arise for several reasons:

- The server merged updated content with existing content
- The server applied business rules and altered the content
- The server does not fully support all the features or possible values of the resource

Note that there is no general-purpose method to make merging with existing content or altering the content by business rules safe or predictable - what is possible, safe and/or required is highly context dependent. These kinds of behaviors may be driven by security considerations. With regard to incomplete support, clients can consult the server's base CapabilityStatement [profile \(capabilitystatement-definitions.html#CapabilityStatement.profile\)](#) references to determine which features or values the server does not support.

The PATCH operation offers some support for making changes to a part of a resource and should be used where a client wishes to change just part of a resource, though transactional integrity issues are still important.

To the degree that the server alters the resource for any of the 3 reasons above, the FHIR server will create implementation consequences for the eco-system that it is part of, which will need to be managed (i.e. it will cost more). For this reason, servers **SHOULD** change the resource as little as possible, given the constraints of the system exposing the FHIR resource. However due to the variability that exists within healthcare, this specification allows that servers **MAY** alter the resource on create/update.

Similarly, to the degree that an implementation context makes special rules about merging content or altering the content, that context will become more expensive to maintain.

Although these rules are stated with regard to servers, a similar concept applies to clients - to the degree that different client systems interacting with the server do not support the same feature set, the clients and/or the server will be forced to implement custom logic to prevent information from being lost or corrupted.

Some of these problems can be mitigated by following a pattern built on top of version-aware updates. In this pattern:

- The server provides a read interaction for any resource it accepts update interactions on
- Before updating, the client read s the latest version of the resource
- The client applies the changes it wants to the resource, leaving other information intact (note the extension related rules ([extensibility.html#exchange](#)) around this)
- The client writes the result back as an update interaction, and is able to handle a **409** or **412** response (usually by trying again)

If clients follow this pattern, then information from other systems that they do not understand will be maintained through the update.

Note that it is possible for a server to choose to maintain the information that would be lost, but there is no defined way for a server to determine whether the client omitted the information because it wasn't supported (perhaps in this case) or whether it wishes to delete the information.

3.1.0.13.1 Conformance

Both client and server systems **SHOULD** clearly document how transaction integrity is handled, in the documentation in the CapabilityStatement ([capabilitystatement.html](#)).

3.1.0.14 Paging

Servers **SHOULD** support paging for the results of a search or history interaction, and if they do, they **SHALL** conform to this method (adapted from [RFC 5005 \(Feed Paging and Archiving\)](#) <https://tools.ietf.org/html/rfc5005>) for sending continuation links to the client when returning a Bundle ([bundle.html](#)) (e.g. with `history` and `search`).

If the server does not do this then there is no way to continue paging.

This example shows the third page of a search result:

```

<Bundle xmlns="http://hl7.org/fhir">
  <!-- snip metadata -->
  <!-- This Search url starts with base search, and adds the effective
    parameters, and additional parameters for search state. All searches
    SHALL return this value.

        In this case, the search continuation method is that the server
        maintains a state, with page references into the stateful list.
    -->
  <link>
    <relation value="self">
    <url value="http://example.org/Patient?name=peter&stateid=23&page=3"/>
  </link>
  <!-- 4 links for navigation in the search. All of these are optional, but recommended -
->

  <link>
    <relation value="first"/>
    <url value="http://example.org/Patient?name=peter&stateid=23&page=1"/>
  </link>
  <link>
    <relation value="previous"/>
    <url value="http://example.org/Patient?name=peter&stateid=23&page=2"/>
  </link>
  <link>
    <relation value="next"/>
    <url value="http://example.org/Patient?name=peter&stateid=23&page=4"/>
  </link>
  <link>
    <relation value="last"/>
    <url value="http://example.org/Patient?name=peter&stateid=23&page=26"/>
  </link>

  <!-- then the search results... -->
</Bundle>

```

The links are opaque to the client, have no dictated structure, and only the server understands them. The client must use the server supplied links in order to traverse the pages. A server MAY add additional state tracking parameters to the links, as shown in the example above, though the server need not use a stateful paging method as shown in this example - it is at the discretion of the server how to best ensure that the continuation retains integrity in the context of ongoing changes to the resources. An alternative approach is to use version specific references to the records on the boundaries, but this is subject to continuity failures when records are updated.

A server MAY inform the client of the total number of resources returned by the interaction for which the results are paged using the [Bundle.total \(bundle-definitions.html#Bundle.total\)](#).

Note that for search, where `_include` can be used to return additional related resources, the total number of resources in the feed may exceed the number indicated in `totalResults`.

In the case of a `search`, the initial request may be made via a POST, but the follow up page requests will be made via GET requests. However servers SHOULD allow for a client to convert the follow up requests to be made via a POST.

3.1.0.15 Support for HEAD

Anywhere that a GET request can be used, a HEAD request is also allowed. HEAD requests are treated as specified in HTTP: same response as a GET, but with no body.

Servers that do not support HEAD MUST respond in accordance with the HTTP specification, for example using a 405 ("method not allowed") or a 501 ("not implemented").

3.1.0.16 Custom Headers

This specification defines or recommends some custom headers that implementers can use to assist with deployment/debugging purposes:

X-Request-Id	A unique id to for the request/response assigned by either client or server. Request: assigned by the client. Response: assigned by the server
X-Correlation-Id	A client assigned request id echoed back in the response
X-Forwarded-For	Identifies the originating IP address of a client to an intermediary
X-Forwarded-Host	Identifies the original host requested by the client in the Host HTTP request header
X-Intermediary	Stamped by an active intermediary that changes the request or the response to alter it's content (see below)

The request id in **X-Request-Id** is purely to help connect between requests and logs/audit trails. The client can assign an id to the request, and send that in the X-Request-Id header. The server can either use that id or assign it's own, which it returns as the **X-Request-Id** header in the response. When the server assigned id is different to the client assigned id, the server **SHOULD** also return the **X-Correlation-Id** header with the client's original id in it.

The HTTP protocol may be routed through an HTTP proxy (e.g. as squid). Such proxies are transparent to the applications, though implementers should be alert to the effects of caching, particularly including the risk of receiving stale content. See the [HTTP specification](http://www.w3.org/Protocols/rfc2616/rfc2616-sec13.html) (<http://www.w3.org/Protocols/rfc2616/rfc2616-sec13.html>) for further detail

Interface engines may also be placed between the consumer and the provider. These differ from proxies because they actively alter the content and/or destination of the HTTP exchange and are not bound by the rules that apply to HTTP proxies. Such agents are allowed, but **SHALL** mark the request with an X-Intermediary header to assist with debugging/troubleshooting. Any agent that modifies an HTTP request or response content other than under the rules for HTTP proxies **SHALL** add a stamp to the HTTP headers like this:

X-Intermediary : [identity - usually a FQDN]

End point systems **SHALL NOT** use this header for any purpose. Its aim is to assist with system troubleshooting.

3.1.0.17 Summary

These tables present a summary of the interactions described here. Note that *all* requests may include an optional **Accept** header to indicate the format used for the response (this is even true for **DELETE** since an OperationOutcome may be returned).

Interaction	Path	Request				
		Verb	Content-Type	Body	Prefer	Conditional
read	/[type]/[id]	GET	N/A	N/A	N/A	O: ETag , If-Modified-Since , If-None-Match
vread	/[type]/[id]/_history/[vid]	GET	N/A	N/A	N/A	N/A
update	/[type]/[id]	PUT	R	Resource	O	O: If-Match
patch	/[type]/[id]	PATCH	R (may be a patch type)	Patch	O	O: If-Match
delete	/[type]/[id]	DELETE	N/A	N/A	N/A	N/A
create	/[type]	POST	R	Resource	O	O: If-None-Exist
search	/[type]?	GET	N/A	N/A	N/A	N/A
	/[type]/_search?	POST	application/x-www-form-urlencoded	form data	N/A	N/A
search-all	?	GET	N/A	N/A	N/A	N/A
capabilities	/metadata	GET	N/A	N/A	N/A	N/A
transaction	/	POST	R	Bundle	O	N/A
history	/[type]/[id]/_history	GET	N/A	N/A	N/A	N/A
history-type	/[type]/_history	GET	N/A	N/A	N/A	N/A
history-all	/_history	GET	N/A	N/A	N/A	N/A
(operation)	/\$[name] , /[type]/[\$[name]] or /[type]/[id]/[\$[name]]	POST	R	Parameters	N/A	N/A
		GET	N/A	N/A	N/A	N/A
		POST	application/x-www-form-urlencoded	form data	N/A	N/A

Notes:

- N/A = not present, R = Required, O = optional
- For operations defined on all resources, including direct access to the meta element, see [Resource Operations \(resource-operations.html\)](#)

Interaction	Response				
	Content-Type	Body	Location	Versioning	Status Codes
read	R	R: Resource	N/A	R: ETag , Last-Modified	200 , 404 , 410
vread	R	R: Resource	N/A	R: ETag , Last-Modified	200 , 404
update	R if body	O: Resource (Prefer)	N/A	R: ETag , Last-Modified	200 , 201 , 400 , 404 , 405 , 409 , 412 , 422
patch	R if body	O: Resource (Prefer)	N/A	R: ETag , Last-Modified	200 , 201 , 400 , 404 , 405 , 409 , 412 , 422
delete	R if body	O: OperationOutcome	N/A	N/A	200 , 202 , 204 , 404 , 405 , 409 , 412
create	R if body	O : Resource (Prefer)	R	R: ETag , Last-Modified	201 , 400 , 404 , 405 , 422
search	R	R: Bundle	N/A	N/A	200 , 401 ?
search-all	R	R: Bundle	N/A	N/A	200 , 401 ?
capabilities	R	R: CapabilityStatement	N/A	N/A	200 , 404
transaction	R	R: Bundle	N/A	N/A	200 , 400 , 404 , 405 , 409 , 412 , 422
history	R	R: Bundle	N/A	N/A	200
history-type	R	R: Bundle	N/A	N/A	200
history-all	R	R: Bundle	N/A	N/A	200
(operation)	R	R: Parameters/Resource	N/A	N/A	200

Note: this table lists the status codes described here, but other status codes are possible as described by the HTTP specification. Additional codes that are likely are server errors and various codes associated with authentication protocols. The [security page \(security.html\)](#) notes several security related issues that may impact which codes to return.

© HL7.org 2011+. FHIR Release 4 (Technical Correction #1) (v4.0.1) generated on Fri, Nov 1, 2019 09:37+1100. [QA Page \(qa.html\)](#)

Links: [Search](#) (<http://hl7.org/fhir/search.cfm>) | [Version History \(history.html\)](#) | [Table of Contents \(toc.html\)](#) | [Credits \(credits.html\)](#) | [Compare to R3](#) (<https://www.fhir.org/perl/htmldiff.pl?oldfile=http%3A%2F%2Fhl7.org%2Ffhir%2FSTU3%2Fhttp.html&newfile=http%3A%2F%2Fhl7.org%2Ffhir%2FR4%2Fhttp.html>) | [Public Domain](#) ([license.html](#)) | [Propose a change](#) (<http://hl7.org/fhir-issues>)